

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

C02: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **MITHRA DEVI S – 192524203** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Mithra Devi

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

1. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
2. Explain how A* would handle the same situation, including how it balances cost and heuristic
3. Analyze the impact of heuristic accuracy on both algorithms
4. Discuss how dynamic changes (blocked paths) affect search performance
5. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

1. Explain how Hill Climbing would work in this scenario and identify its limitations
2. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
3. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
4. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

1. Explain why this problem requires an online search agent instead of offline search
2. Analyze the challenges of partial observability and dynamic environments
3. Describe how the rover builds and updates its internal model
4. Compare online search with uninformed and informed search strategies
5. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

1. Clearly define variables, domains, and constraints with explanation
2. Explain how backtracking search works in this scenario
3. Discuss how constraint propagation (e.g., forward checking) improves efficiency
4. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

1. Explain how the Minimax algorithm models this problem
2. Analyze the computational challenges of large game trees
3. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
4. Design an evaluation function and justify the factors considered
5. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1

Scenario: AI-Powered Drone for Emergency Medicine Delivery

Scenario Understanding

A government uses AI-powered drones to deliver emergency medicines in flood-affected areas. The environment changes continuously because roads may become blocked due to floods, weather conditions may worsen, and different terrains increase travel cost. The drone has only partial real-time information and must choose the safest and fastest route to save lives. Therefore, an intelligent search algorithm is required to make quick and accurate decisions.

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Working

Greedy Best-First Search (GBFS) chooses the next path based only on the heuristic value $h(n)$, which estimates the distance from the current location to the destination.

Evaluation Function:

$$f(n) = h(n)$$

The drone always selects the path that appears closest to the destination without considering the actual travel cost.

Example

Suppose the drone has three possible routes.

Route	Heuristic Distance
Route A	8 km
Route B	5 km
Route C	10 km

Greedy Search immediately selects **Route B** because it has the smallest heuristic distance.

Strengths

- Makes decisions very quickly.
- Requires less computation.
- Suitable for emergency situations where immediate action is needed.

- Easy to implement.
- Performs well when the heuristic estimate is accurate.

Weaknesses under Uncertainty

- Ignores actual travel cost.
 - May choose flooded or blocked roads.
 - Cannot guarantee the shortest or safest route.
 - Can become trapped in poor paths.
 - Frequently requires replanning when conditions change.
-

*ii. Explain how A would handle the same situation, including how it balances cost and heuristic.**

Working

A* Search considers both:

- Actual cost from the start **$g(n)$**
- Estimated remaining cost **$h(n)$**

Evaluation Function:

$$f(n) = g(n) + h(n)$$

Instead of simply choosing the closest route, A* selects the path with the lowest total estimated cost.

Example

Route $g(n)$ $h(n)$ $f(n)=g+h$

A 4 6 10

B 7 2 9

C 3 8 11

Although Route A has a shorter actual distance than B, A* chooses **Route B** because its total estimated cost is the lowest.

Advantages

- Produces an optimal path when the heuristic is admissible.
 - Considers both safety and travel time.
 - Avoids unnecessarily expensive routes.
 - Adapts better to changing environments.
 - More reliable for disaster management.
-

Disadvantages

- Requires more memory.
 - Computationally slower than Greedy Search.
 - Needs frequent updates when the environment changes.
-

iii. Analyze the impact of heuristic accuracy on both algorithms.

The heuristic function estimates the remaining distance to the goal.

When heuristic is highly accurate

- Greedy Search quickly reaches the destination.
- A* finds the optimal route with fewer node expansions.
- Search time decreases.

When heuristic is inaccurate

- Greedy Search may move toward blocked or dangerous roads.
- A* still performs better because it also considers the actual path cost.
- More nodes are explored before finding the solution.

Comparison

Heuristic Quality	Greedy Search	A* Search
Accurate	Very fast	Fast and optimal
Moderately accurate	May miss good routes	Usually finds good solution
Poor	Performs poorly	Still reliable but slower

iv. Discuss how dynamic changes (blocked paths) affect search performance.

Flood situations change rapidly.

Examples:

- Roads become submerged.
- Bridges collapse.
- Heavy rainfall increases travel time.
- Wind slows drone movement.

Effect on Greedy Search

- Frequently chooses blocked paths.
- Needs repeated replanning.
- May waste valuable time.

Effect on A*

- Updates route after receiving new information.
- Searches for an alternative lower-cost path.
- More robust in dynamic environments.

Although both algorithms require replanning, A* generally recovers more effectively because it evaluates total cost instead of only the heuristic estimate.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Recommendation: A Search Algorithm*

A* is the most suitable algorithm for emergency medicine delivery.

Justification

Real-Time Performance

- Makes intelligent routing decisions.
- Can incorporate updated information.
- Responds effectively to changing conditions.

Optimality

- Finds the minimum-cost path.
- Avoids unnecessary delays.

- Reduces travel distance and fuel consumption.

Safety

- Avoids dangerous or flooded areas.
- Considers both travel cost and destination distance.
- Improves the reliability of emergency deliveries.

Comparison

Feature	Greedy Search	A* Search
Speed	Very Fast	Fast
Optimal Solution	No	Yes
Uses Actual Cost	No	Yes
Handles Dynamic Changes	Moderate	Excellent
Safety	Moderate	High
Suitable for Disaster Relief	Partially	Highly Suitable

Conclusion

The emergency drone operates in a highly dynamic and uncertain environment where both speed and safety are essential. Greedy Best-First Search provides quick decisions but may select unsafe or inefficient routes because it considers only the heuristic estimate. In contrast, A* Search combines the actual travel cost with the heuristic estimate to find the safest and most efficient path. Its ability to adapt to changing conditions and produce near-optimal routes makes it the best choice for emergency medicine delivery in flood-affected regions. This improves response time, reduces risk, and ensures reliable healthcare support during disasters.

QUESTION 2

Scenario: Smart City Traffic Signal Optimization

Scenario Understanding

A smart city uses an AI-based traffic management system to optimize traffic signal timings across hundreds of intersections. The main objectives are to reduce total waiting time, fuel consumption, and traffic congestion. Since traffic conditions change continuously throughout the day, the AI system must quickly adjust signal timings. The search space is extremely large, making traditional search methods inefficient. Therefore, optimization algorithms such as Hill Climbing, Simulated Annealing, and Genetic Algorithms are suitable for solving this problem.

i. Formulate this as an optimization problem and explain why traditional search is inefficient.

Optimization Problem

The AI system aims to find the **best traffic signal timing** for every intersection.

Objective Function

Minimize:

- Total waiting time of vehicles.
- Fuel consumption.
- Traffic congestion.
- Number of traffic jams.

Decision Variables

- Green signal duration.
- Red signal duration.
- Yellow signal duration.
- Signal sequence at each intersection.

Constraints

- Every road must receive sufficient green time.
- Emergency vehicles should get priority.
- Pedestrian crossing time must be ensured.
- Traffic rules must be followed.

Why Traditional Search is Inefficient

Traditional search algorithms like Breadth-First Search (BFS) or Depth-First Search (DFS) explore a very large number of possible signal combinations.

In a smart city:

- Hundreds of intersections exist.
- Each signal has multiple timing options.
- Traffic changes every minute.

This creates an enormous search space, making traditional search:

- Very slow.
- Memory intensive.
- Unsuitable for real-time optimization.

Hence, optimization algorithms are preferred.

ii. Explain how Hill Climbing would work in this scenario and identify its limitations.

Working of Hill Climbing

Hill Climbing starts with an initial traffic signal schedule and evaluates its performance.

Steps:

1. Start with current signal timings.
2. Calculate the waiting time.
3. Slightly modify one signal timing.
4. Compare the new solution with the current one.
5. If the new solution is better, move to it.
6. Repeat until no further improvement is possible.

Example

Current waiting time = **600 seconds**

After increasing one green signal:

New waiting time = **540 seconds**

Since the waiting time decreases, the AI accepts the new solution.

Advantages

- Very simple algorithm.
 - Fast execution.
 - Requires less memory.
 - Suitable for small optimization problems.
-

Limitations

- Can get stuck at local optimum.
 - Cannot always find the best solution.
 - Sensitive to the starting solution.
 - Performs poorly in highly dynamic traffic conditions.
-

iii. Discuss issues like Local Maxima, Plateaus, and Ridges with real-world interpretation.

1. Local Maxima (Local Optimum)

A local optimum is a solution that appears to be the best nearby but is not the overall best solution.

Traffic Example

Improving one intersection reduces waiting time locally, but overall city traffic remains congested.

2. Plateau

A plateau occurs when many neighboring solutions have the same evaluation value.

Traffic Example

Changing signal timings by a few seconds produces no improvement in traffic flow.

The algorithm cannot determine which direction to move.

3. Ridge

A ridge occurs when improvement requires multiple coordinated changes rather than a single small adjustment.

Traffic Example

Reducing congestion requires changing signal timings at several nearby intersections simultaneously.

Hill Climbing changes only one variable at a time, making progress difficult.

Summary

Problem	Real-World Interpretation
----------------	----------------------------------

Local Maxima AI thinks it has found the best solution, but better solutions exist elsewhere.

Plateau	Many signal timings give identical performance.
---------	---

Ridge	Improvement requires multiple coordinated signal changes.
-------	---

iv. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing

Simulated Annealing occasionally accepts worse solutions to escape local optima.

Working

- Starts with a high "temperature."
- Sometimes accepts poorer solutions.
- Temperature gradually decreases.
- Eventually converges to a high-quality solution.

Advantages

- Escapes local maxima.
 - Explores more possible solutions.
 - Produces better optimization than Hill Climbing.
-

Genetic Algorithm (GA)

Genetic Algorithm is inspired by natural evolution.

Working

1. Generate many traffic signal schedules.
 2. Evaluate each schedule.
 3. Select the best schedules.
 4. Perform crossover.
 5. Apply mutation.
 6. Generate improved schedules.
 7. Repeat until the best solution is found.
-

Advantages of Genetic Algorithm

- Explores a very large search space.
 - Avoids local optima.
 - Works well for complex traffic systems.
 - Can optimize multiple objectives simultaneously.
 - Easily adapts to changing traffic patterns.
-

Comparison

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes Local Optimum	No	Yes	Yes
Global Search	No	Partial	Excellent
Suitable for Large Search Space	Limited	Good	Excellent
Adaptability	Low	Medium	High

v. Recommend the best approach and justify your choice based on scalability and adaptability.

Recommendation: Genetic Algorithm

Genetic Algorithm is the most suitable approach for smart city traffic optimization.

Justification

Scalability

- Efficiently handles hundreds of intersections.
- Processes thousands of possible signal schedules.
- Suitable for large-scale optimization problems.

Adaptability

- Continuously adjusts signal timings based on traffic conditions.
- Responds effectively to changing traffic flow.
- Optimizes multiple objectives simultaneously.

Efficiency

- Reduces waiting time.
- Minimizes fuel consumption.
- Lowers traffic congestion.
- Improves overall transportation efficiency.

Comparison

Criteria	Hill Climbing Simulated Annealing Genetic Algorithm		
Speed	Fast	Moderate	Moderate
Solution Quality	Moderate	High	Very High
Scalability	Low	Medium	Excellent
Adaptability	Low	High	Excellent
Suitable for Smart City	Partially	Good	Highly Suitable

Conclusion

Traffic signal optimization is a complex real-time optimization problem with a very large search space. Traditional search algorithms are inefficient because they cannot handle the enormous number of possible signal combinations. Hill Climbing provides a simple and fast solution but often gets trapped in local optima. Simulated Annealing improves performance by escaping local optima, while the **Genetic Algorithm** offers the best balance of scalability, adaptability, and optimization quality. Therefore, the **Genetic Algorithm is the most suitable approach** for managing smart city traffic, as it minimizes congestion, reduces waiting time and fuel consumption, and continuously adapts to changing traffic conditions.

QUESTION 3

Scenario: Autonomous Mars Rover – Online Search Agent

Scenario Understanding

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. It must navigate safely while avoiding hazards such as rocks and craters. Since the rover has only limited sensing capability and no complete map of the environment, it must continuously update its knowledge and make decisions in real time. Therefore, an **Online Search Agent** is the most suitable approach for this scenario.

i. Explain why this problem requires an Online Search Agent instead of Offline Search.

Online Search Agent

An Online Search Agent makes decisions while exploring the environment. It does not require complete knowledge of the map before starting.

Why Online Search is Needed

- The rover does not have a complete map of Mars.
- New obstacles are discovered during exploration.
- The environment changes continuously.
- The rover must make immediate decisions.
- It learns from newly collected information.

Why Offline Search is Not Suitable

Offline Search requires:

- Complete map of the environment.
- All obstacles known in advance.
- Entire path planned before movement.

These conditions are not possible on Mars because the terrain is unknown.

Comparison

Online Search	Offline Search
Works in unknown environments	Requires complete knowledge
Learns while moving	Plans before moving
Adapts to new obstacles	Cannot easily adapt
Suitable for Mars rover	Not suitable for Mars exploration

ii. Analyze the challenges of partial observability and dynamic environments.

1. Partial Observability

The rover can only observe nearby areas using its sensors.

Challenges

- Unknown obstacles beyond sensor range.
- Hidden craters or rocks.
- Limited visibility due to dust storms.
- Incomplete information about terrain.

2. Dynamic Environment

The environment may change while the rover is operating.

Examples

- Dust storms reduce visibility.
- Loose rocks may shift.
- Sand dunes may change shape.
- Weather conditions may affect movement.

Impact

- Previously safe paths may become unsafe.
- The rover must frequently update its route.
- Decision-making becomes more complex.

iii. Describe how the rover builds and updates its internal model.

The rover maintains an **internal model** of the environment.

Working Process

1. Start with limited knowledge.
2. Move to a nearby location.
3. Use sensors to scan surroundings.
4. Detect obstacles and safe paths.
5. Update the internal map.

6. Store new information.
7. Plan the next movement using the updated map.
8. Repeat until the mission is completed.

Internal Model Stores

- Safe paths.
- Dangerous areas.
- Rock locations.
- Craters.
- Sample collection points.
- Distance traveled.

Advantages

- Improves navigation accuracy.
- Reduces repeated exploration.
- Helps avoid hazards.
- Increases mission efficiency.

iv. Compare Online Search with Uninformed and Informed Search strategies.

Feature	Online Search		
	Uninformed Search	Informed Search	
Environment Knowledge	Partial	None	Uses heuristic information
Complete Map Required	No	Usually Yes	Usually Yes
Learns During Search	Yes	No	No
Adapts to New Obstacles	Yes	Limited	Limited
Decision Making	Real-time	Predefined	Heuristic-based
Suitable for Mars Rover	Excellent	Poor	Moderate

Explanation

Uninformed Search

- Explores without heuristic information.
- May waste time exploring unnecessary paths.

- Inefficient in unknown environments.

Informed Search

- Uses heuristic estimates to guide the search.
- Performs better than uninformed search.
- Still assumes sufficient knowledge of the environment.

Online Search

- Learns while exploring.
 - Continuously updates knowledge.
 - Best suited for unknown and changing environments.
-

v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Recommendation: Online Search Agent

The Online Search Agent is the most suitable approach for the Mars rover.

Justification

Uncertainty

- Handles unknown terrain effectively.
- Makes decisions using available sensor data.
- Does not require a complete map.

Adaptability

- Updates routes when new obstacles are detected.
- Learns from exploration.
- Continuously improves navigation.

Safety

- Avoids rocks and craters.
- Reduces collision risk.
- Chooses safer paths using updated information.

Efficiency

- Saves energy by avoiding unnecessary movement.
- Improves mission success rate.

- Supports long-duration autonomous exploration.

Comparison

Criteria	Offline Search	Online Search
Unknown Environment	No	Yes
Real-Time Decision Making	No	Yes
Adaptability	Low	High
Safety	Moderate	High
Mars Exploration	Not Suitable	Highly Suitable

Conclusion

The Mars rover operates in an uncertain and partially observable environment where a complete map is unavailable. Offline search methods are unsuitable because they require full knowledge of the environment before planning. An **Online Search Agent** continuously senses its surroundings, updates its internal model, and adapts its decisions in real time. This approach enables the rover to navigate safely, avoid hazards, collect scientific samples efficiently, and successfully complete its mission. Therefore, the **Online Search Agent** is the most appropriate solution for autonomous Mars exploration.

QUESTION 4

Scenario: University Exam Timetable Generation using Constraint Satisfaction Problem (CSP)

Scenario Understanding

A university wants to automatically generate an exam timetable using Artificial Intelligence. The timetable must satisfy several constraints such as preventing overlapping exams for students, managing limited exam halls, ensuring faculty availability, scheduling prerequisite subjects before others, and providing special accommodations for some students. As the number of courses and students increases, the scheduling problem becomes highly complex. Therefore, it is best modeled as a **Constraint Satisfaction Problem (CSP)**.

i. Formulate the problem as a Constraint Satisfaction Problem (CSP). Clearly define variables, domains, and constraints with explanation.

Constraint Satisfaction Problem (CSP)

A CSP consists of:

- **Variables**
- **Domains**
- **Constraints**

Variables

Each course is represented as a variable.

Example:

- C1 = Mathematics
- C2 = Physics
- C3 = Chemistry
- C4 = Computer Science
- C5 = English

Each variable represents the exam time assigned to that course.

Domains

The domain is the set of available exam slots.

Example:

- Day 1 – Morning
- Day 1 – Afternoon
- Day 2 – Morning
- Day 2 – Afternoon
- Day 3 – Morning

Each course must be assigned one valid time slot.

Constraints

The timetable must satisfy the following constraints:

1. No student should have overlapping exams.
 2. Limited exam halls must not be exceeded.
 3. Some subjects must be conducted before others.
 4. Faculty members must be available during the assigned exam time.
 5. Students requiring special accommodations should receive suitable arrangements.
 6. Every course must be assigned exactly one time slot.
-

Example

Course	Assigned Time Slot
Mathematics	Day 1 Morning
Physics	Day 1 Afternoon
Chemistry	Day 2 Morning
Computer Science	Day 2 Afternoon
English	Day 3 Morning

All constraints are satisfied because there are no clashes, halls are available, and faculty schedules are respected.

ii. Explain how Backtracking Search works in this scenario.

Working of Backtracking Search

Backtracking Search assigns time slots to exams one by one.

Steps

1. Select an unassigned course.
 2. Assign an available time slot.
 3. Check whether all constraints are satisfied.
 4. If no constraint is violated, assign the next course.
 5. If a conflict occurs, remove the assignment (backtrack).
 6. Try another available time slot.
 7. Repeat until all courses are scheduled successfully.
-

Example

- Assign Mathematics → Day 1 Morning ✓
- Assign Physics → Day 1 Morning ✗ (Student conflict)
- Backtrack
- Assign Physics → Day 1 Afternoon ✓
- Continue assigning remaining courses.

This process continues until a valid timetable is obtained.

Advantages

- Guarantees a valid solution if one exists.
 - Easy to implement.
 - Systematically explores possible assignments.
 - Avoids invalid schedules.
-

iii. Discuss how Constraint Propagation (Forward Checking) improves efficiency.

Forward Checking

Forward Checking reduces the search space by checking future assignments immediately after assigning a value.

Working

After assigning a time slot to one course:

- Remove conflicting time slots from other courses.
- Detect conflicts early.
- Prevent unnecessary exploration.
- Reduce backtracking.

Example

Suppose:

- Mathematics → Day 1 Morning

Then any course with common students cannot use **Day 1 Morning**.

That option is removed from their domains.

This prevents future conflicts and speeds up the scheduling process.

Advantages of Forward Checking

- Detects conflicts early.
 - Reduces unnecessary searches.
 - Minimizes backtracking.
 - Improves execution speed.
 - Suitable for large scheduling problems.
-

iv. Analyze real-world challenges and justify the best strategy for scalability.

Real-World Challenges

1. Large number of students.
2. Hundreds of courses.
3. Limited exam halls.
4. Faculty availability changes.
5. Last-minute timetable modifications.
6. Special accommodations for differently-abled students.
7. Multiple departments sharing resources.

These factors make timetable generation a highly complex optimization problem.

Best Strategy

Backtracking Search + Forward Checking (Constraint Propagation)

This combination is the most suitable approach.

Justification

Scalability

- Efficiently handles a large number of courses and students.
- Reduces unnecessary computations.
- Solves complex scheduling problems effectively.

Efficiency

- Detects conflicts early.
- Minimizes backtracking.
- Produces valid timetables faster.

Reliability

- Ensures all constraints are satisfied.
 - Reduces scheduling errors.
 - Generates high-quality exam schedules.
-

Comparison

Criteria	Backtracking Only	Backtracking + Forward Checking
Speed	Moderate	High
Backtracking	More	Less
Conflict Detection	Late	Early
Scalability	Moderate	Excellent
Suitable for University Timetabling	Good	Highly Suitable

Conclusion

University exam timetable generation is a complex **Constraint Satisfaction Problem (CSP)** involving multiple variables, domains, and constraints. **Backtracking Search** systematically assigns exam slots while ensuring all constraints are satisfied. Adding **Forward Checking (Constraint Propagation)** significantly improves efficiency by detecting conflicts early and reducing unnecessary backtracking. Therefore, the combination of **Backtracking Search with Forward Checking** is the most suitable strategy for generating scalable, accurate, and conflict-free university exam timetables.

QUESTION 5

Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Scenario Understanding

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players by making intelligent decisions within a limited time. The game has many possible moves, creating a very large decision tree. The AI must evaluate different game states and choose the best move quickly while maintaining good performance. Therefore, **Minimax** and **Alpha-Beta Pruning** are the most suitable algorithms.

i. Explain how the Minimax algorithm models this problem.

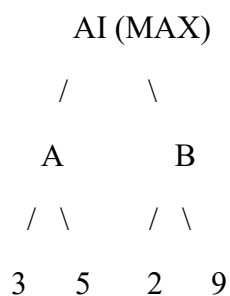
Minimax Algorithm

Minimax is a decision-making algorithm used in two-player games where one player tries to **maximize** the score while the opponent tries to **minimize** it.

Working

1. Generate all possible moves.
2. Build a game tree.
3. Evaluate each game state.
4. Assume the opponent always plays the best move.
5. Choose the move that gives the highest guaranteed score.

Example



- Move **A** → **Min value** = 3
- Move **B** → **Min value** = 2

The AI chooses **Move A**, because **MAX selects the higher value (3)**.

Advantages

- Makes intelligent decisions.
 - Considers future moves.
 - Guarantees the best move if the entire tree is explored.
 - Suitable for turn-based strategy games.
-

ii. Analyze the computational challenges of large game trees.

In real-time strategy games:

- Thousands of possible moves exist.
- Every move creates many new game states.
- The search tree grows exponentially.

Challenges

- Very high computation time.
- Large memory requirement.
- Difficult to search the complete tree.
- Delays decision making.
- Not practical for real-time games without optimization.

Example

If:

- Branching Factor = 30
- Search Depth = 6

Total nodes:

$$30^6 = 729,000,000 \text{ nodes}$$

Searching all nodes is extremely expensive.

iii. Explain how Alpha-Beta Pruning improves efficiency with detailed reasoning.

Alpha-Beta Pruning

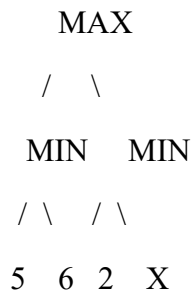
Alpha-Beta Pruning is an optimization of the Minimax algorithm.

It removes branches that cannot affect the final decision.

Working

- **Alpha (α)** = Best value found for the MAX player.
- **Beta (β)** = Best value found for the MIN player.
- If $\alpha \geq \beta$, the remaining branches are **pruned** because they cannot improve the result.

Example



After evaluating the left branch, the AI knows that the right branch cannot produce a better result. Therefore, node **X** is **not evaluated**.

Advantages

- Reduces the number of nodes explored.
- Produces the same optimal result as Minimax.
- Faster decision making.
- Lower memory usage.
- Suitable for real-time games.

iv. Design an evaluation function and justify the factors considered.

An **evaluation function** estimates the quality of a game state when the search cannot continue to the end of the game.

Example Evaluation Function

Score = (5 × Resources) + (10 × Army Strength) + (15 × Strategic Position) – (8 × Damage Taken)

Factors Considered

1. **Resources**
 - Gold, food, and energy available.

- More resources allow stronger future actions.

2. Army Strength

- Number and power of military units.
- Stronger armies increase the chance of winning.

3. Strategic Position

- Control of important locations or objectives.
- Better positions provide tactical advantages.

4. Damage Taken

- Health lost by units or structures.
- Lower damage improves survival and future performance.

Why These Factors?

- Reflect the overall strength of the AI.
 - Help estimate future winning chances.
 - Enable quick and effective decision-making without exploring the entire game tree.
-

v. Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Minimax with Alpha-Beta Pruning

This combination is the most suitable strategy for real-time strategy games.

Justification

Optimality

- Minimax selects the best possible move assuming an intelligent opponent.
- Improves the AI's chances of winning.

Real-Time Performance

- Alpha-Beta Pruning reduces unnecessary node evaluations.
- Faster decisions are made within strict time limits.

Efficiency

- Lower computation time.
- Reduced memory usage.
- Allows deeper search within the same time.

Scalability

- Handles larger game trees efficiently.
- Suitable for complex real-time strategy games.

Comparison

Criteria	Minimax	Minimax + Alpha-Beta Pruning
Optimal Solution	Yes	Yes
Nodes Explored	Very High	Much Lower
Execution Speed	Slow	Fast
Memory Usage	High	Lower
Suitable for Real-Time Games	Limited	Highly Suitable

Conclusion

Real-time strategy games require the AI to make intelligent decisions quickly despite a very large search space. **Minimax** provides optimal decision-making by considering both the AI's and the opponent's moves, but it becomes computationally expensive as the game tree grows. **Alpha-Beta Pruning** significantly improves efficiency by eliminating branches that cannot influence the final decision, reducing computation time while preserving optimality. Therefore, the combination of **Minimax with Alpha-Beta Pruning**, supported by an effective evaluation function, is the best approach for developing a fast, intelligent, and competitive game AI.